

JZ-HDL SIMULATION SPECIFICATION

State: Beta — Version: 0.2.0

Contents

1. OVERVIEW	3
2. TIMING AND EXECUTION MODEL	4
2.1 Internal Time Resolution	4
2.2 Event-Driven Clock Scheduling	4
2.2.1 Clock Jitter	4
2.2.2 Clock Drift	5
2.3 Time 0 Initialization	6
2.4 Execution Flow	7
2.4.1 Determinism Guarantee	7
2.6 Combinational Settling	7
2.5 X and Z Semantics	7
3. SIMULATION STRUCTURE	9
3.1 Declaration	9
3.2 CLOCK Block (Auto-toggling)	9
3.3 WIRE Block	9
3.4 TAP Block (Internal Monitoring)	9
4. STIMULUS AND CONTROL DIRECTIVES	11
4.1 @setup	11
4.2 @run (Time Advancement)	11
4.3 @update	11
4.4 @run_until (Conditional Time Advancement)	11
4.5 @run_while (Conditional Time Advancement)	12
4.6 @repeat	12
4.7 @print	13
4.8 @print_if	14
4.9 @trace (Waveform Recording Control)	14
4.10 @mark (Waveform Marker)	15
4.11 @alert (Unconditional Alert Annotation)	16
4.12 @mark_if (Conditional One-Shot Marker)	17
4.13 @alert_if (Conditional One-Shot Alert)	18
4.14 MONITOR Block (Per-Tick Conditional Evaluation)	18
5. CLI USAGE	21
5.1 --simulate	21
5.2 Flags	21
6. WAVEFORM OUTPUT	23
6.1 VCD Format	23
6.2 Signal Grouping	23
6.3 What Gets Recorded	23
6.4 JZW Metadata	23
7. EXAMPLE	24

1. OVERVIEW

This specification defines the `@simulation` construct for JZ-HDL. While `@testbench` provides strictly manual, cycle-stepped control for functional verification, `@simulation` provides a **time-based, multi-clock continuous simulation environment**.

In a `@simulation`, clocks run automatically in the background based on their defined periods. The simulator uses event-driven scheduling with 1 picosecond resolution, jumping directly between clock edges. The test sequence advances via absolute time (`@run`) directives rather than manual edge toggles. All declared `WIRES`, `CLOCKS`, and `TAPped` internal signals are automatically monitored and dumped to an output waveform file (VCD, FST, or JZW) at each clock event.

Relationship to `@testbench`:

`@simulation` and `@testbench` are two **independent execution models**. They share the same RTL evaluation semantics (combinational settling, NBA for synchronous updates, reset behavior, x/z rules) but differ fundamentally in how time advances:

- `@simulation` is **time-based**: clocks have defined periods and toggle independently via event-driven scheduling at 1ps resolution. Time advances via `@run` directives in absolute units (nanoseconds, milliseconds). Multiple clocks run concurrently.
- `@testbench` is **cycle-stepped**: clocks advance only when explicitly commanded by `@clock` directives, one clock at a time. There is no notion of absolute time or clock periods.

Neither model is a superset of the other. A simulation implementation is not required to use the testbench engine internally, and vice versa. The simulation model is designed for multi-clock timing verification and waveform capture; the testbench model is designed for deterministic, assertion-driven functional tests.

2. TIMING AND EXECUTION MODEL

2.1 Internal Time Resolution

All time values are represented internally as **64-bit unsigned integers in picoseconds**. When a clock period or `@run` duration is specified in nanoseconds or milliseconds, the compiler converts it to an integer picosecond count immediately at parse time. All subsequent arithmetic — GCD computation, tick advancement, clock scheduling — uses pure integer math. This eliminates floating-point drift.

Exact conversion required. The specified value must convert to an **exact integer number of picoseconds**. If the conversion would produce a fractional picosecond, the compiler rejects the value with a diagnostic error. There is no silent rounding or truncation. Examples: - `period=10.0` → 10000 ps (valid) - `period=3.333` → 3333 ps (valid: $3.333 \times 1000 = 3333.0$ exactly) - `ns=0.1` → 100 ps (valid: $0.1 \times 1000 = 100.0$ exactly) - `period=3.3335` → 3333.5 ps (rejected: not an integer picosecond count)

2.2 Event-Driven Clock Scheduling

Unlike `@testbench`, which is strictly cycle-relative, `@simulation` operates on an absolute timeline with **1 picosecond resolution**.

The simulator uses **event-driven scheduling** for clock toggles. Each clock maintains a `next_toggle_ps` timestamp indicating when it will next change value. At initialization, `next_toggle_ps` is set to `phase_ps + toggle_ps` (i.e., the phase offset plus half the period). When advancing time, the simulator jumps directly to the earliest `next_toggle_ps` across all clocks — skipping empty time where no clock toggles. After toggling a clock, its `next_toggle_ps` advances by `toggle_ps`.

This approach provides exact 1ps timing resolution while maintaining $O(\text{number of clock edges})$ performance regardless of the time span being simulated.

2.2.1 Clock Jitter

The simulator supports optional **period jitter** on any clock, modeling the cycle-to-cycle timing variation present in real oscillators and PLLs. Jitter is enabled per clock via the `--jitter` command-line flag (see Section 5.2).

Jitter model:

- **Type: Period jitter (no drift).** Each clock edge is perturbed relative to its **ideal** position. The simulator tracks the ideal `next_toggle_ps` (accumulated from the exact `toggle_ps` half-period) and applies a random offset to produce the actual toggle time. Because the offset is computed from the ideal position — not the previous actual position — jitter does not accumulate into drift. Over long simulations, the clock’s average frequency remains exact.
- **Distribution: Gaussian, clamped.** The jitter offset is drawn from a Gaussian (normal) distribution. The `--jitter` parameter specifies the **peak-to-peak** jitter in picoseconds. The standard deviation is $= \text{peak_to_peak} / 6$, placing the $\pm \text{peak_to_peak} / 2$ bounds at ± 3 . This means 99.7% of edges fall within the specified range naturally, and the remaining 0.3% are clamped to $\pm \text{peak_to_peak} / 2$ to prevent physically unrealistic outliers.

For example, `--jitter=clk:200` specifies 200ps peak-to-peak jitter on clock `clk`: 33.3ps, 99.7% of edges within ± 100 ps of ideal, hard-clamped at ± 100 ps.

- **PRNG: Deterministic per-clock.** Each jittered clock receives its own xorshift32 PRNG state, seeded deterministically from the simulation seed and the clock's declaration index: `jitter_rng = seed ^ (clock_index + 0x4A495454)`. Gaussian samples are generated using the Box-Muller transform on pairs of uniform xorshift32 outputs. This ensures jitter sequences are fully reproducible given the same `--seed` and `--jitter` flags, regardless of platform or implementation.
- **Minimum edge spacing.** If a jitter offset would cause `next_toggle_ps` to be less than or equal to `current_time_ps` (i.e., the edge would move into the past), the offset is clamped so the edge occurs at `current_time_ps + 1`. This prevents negative time advancement and maintains simulation causality.

Interaction with other features:

- Jitter does not affect the GCD tick computation. The GCD is computed from ideal (unjittered) toggle intervals and is used only for `@run(ticks=N)` conversion.
- Jitter does not affect `@run` duration accounting. The simulation runs until the requested wall-clock time has elapsed, regardless of how many jittered edges occur.
- Jitter metadata is recorded in JZW output (see Section 6).

GCD tick for `@run(ticks=N)`: The simulator also computes the Greatest Common Divisor (GCD) of all clock toggle intervals. This GCD defines the **tick unit** used when `@run(ticks=N)` is specified — the duration advanced is $N \times \text{GCD}$ picoseconds. For example, if `clk_a` has a period of 10ns (toggles every 5000ps) and `clk_b` has a period of 14ns (toggles every 7000ps), the GCD is 1000ps = 1ns, and `@run(ticks=10)` advances 10ns. The `@run(ns=...)` and `@run(ms=...)` forms are unaffected — they convert directly to picoseconds.

2.2.2 Clock Drift

The simulator supports optional **frequency drift** on any clock, modeling the crystal tolerance (ppm accuracy) of real oscillators. Drift is enabled per clock via the `--drift` command-line flag (see Section 5.2).

Drift model:

- **Type: Fixed frequency offset.** At simulation start, each drifted clock receives a fixed frequency offset (in parts per million) that persists for the entire simulation. This models the static frequency error of a real crystal oscillator operating at a particular temperature and voltage. Unlike jitter, drift **accumulates** — the clock runs consistently faster or slower than its nominal frequency, causing its edges to progressively diverge from ideal timing.
- **Selection: Gaussian, clamped.** The actual drift value is drawn from a Gaussian distribution at simulation initialization. The `--drift` parameter specifies the **maximum drift** in ppm. The standard deviation is $= \text{max_ppm} / 3$, placing the $\pm \text{max_ppm}$ bounds at ± 3 . Values beyond $\pm \text{max_ppm}$ are clamped. The selected value may be positive (clock runs fast) or negative (clock runs slow).

For example, `--drift=clk:50` specifies ± 50 ppm maximum drift on clock `clk`: 16.7 ppm, 99.7% of runs select a drift within ± 50 ppm naturally, hard-clamped at ± 50 ppm.

- **Implementation: Adjusted half-period.** The drift is applied by computing a drifted half-period: `drifted_toggle_ps = toggle_ps * (1 + actual_ppm / 1,000,000)`. The simulator tracks the ideal next toggle time as a `double` to accumulate sub-picosecond fractional drift without rounding error. Each scheduled toggle rounds the accumulated double to the nearest integer picosecond.
- **PRNG: Deterministic per-clock.** Each drifted clock's actual ppm is selected using a dedicated xorshift32 PRNG state, seeded from the simulation seed and the clock's declaration index: `drift_rng = seed ^ (clock_index + 0x44524654)`. This ensures drift selection is fully reproducible given the same `--seed` and `--drift` flags.

Interaction with jitter:

- When both jitter and drift are active on a clock, drift changes the base period (via `drifted_toggle_ps`), and jitter adds random perturbation on top. The ideal next toggle accumulates using the drifted period, and jitter offsets are applied relative to that drifted ideal position.
- Drift does not affect the GCD tick computation. The GCD is computed from nominal (undrifted) toggle intervals and is used only for `@run(ticks=N)` conversion.
- Drift does not affect `@run` duration accounting. The simulation runs until the requested wall-clock time has elapsed.
- Drift metadata is recorded in JZW output (see Section 6.4).

2.3 Time 0 Initialization

In hardware, clocks do not instantly start toggling the picosecond power is applied. The simulator models this with a deterministic initialization sequence at Time 0, before any clock edge occurs:

1. **Storage randomizes.** All REGISTER, LATCH, and MEM storage is filled with pseudo-random values derived from the simulation seed (`--seed`). This models the indeterminate power-on state of real flip-flops and memory cells.
2. **Clocks hold at 1'b0.** No clock has toggled yet. Clocks with a phase offset (future feature) would use their specified initial value.
3. **@setup executes.** Explicit initial wire values (e.g., `rst_n <= 1'b0`) are applied to the testbench wires.
4. **Combinational logic settles.** Inputs propagate through the DUT, all combinational paths evaluate to a fixed point, and outputs propagate back to testbench wires.
5. **Time 0 is written to the waveform.** The VCD/FST file records the complete initial state — randomized registers, clock-low, setup values, and settled combinational outputs — before any clock edge fires.
6. **The first @run begins advancing ticks.**

Important: A register's declared reset value is never applied automatically at instantiation or power-on. Reset values take effect only when the module's reset signal is explicitly asserted during simulation. If a module has no reset port, or if the simulation never asserts reset, all registers retain their random power-on values for the entire simulation — exactly as in real hardware.

This ensures the waveform viewer shows the full initial setup state before the first clock edge.

2.4 Execution Flow

1. **Time Advancement (@run):** The simulator advances time by jumping to successive clock events up to the requested duration.
2. **Clock Event Evaluation:** At each clock event (a time where one or more clocks toggle):
 - All clocks scheduled to toggle at this time update their values. Each clock's `next_toggle_ps` advances by its `toggle_ps`.
 - Combinational logic evaluates to a fixed point (see Section 2.6).
 - Synchronous assignments are sampled and applied (NBA semantics).
 - Combinational logic settles again.
 - Monitored signals are sampled and written to the output file.
3. **Procedural Updates (@update):** Executed instantaneously between `@run` commands. They apply wire changes, propagate inputs, settle combinational logic, propagate outputs, and record the changes to the waveform before the next `@run` begins.

2.4.1 Determinism Guarantee

Given identical input source files and seed, the simulation must produce bit-identical output — waveform files and all diagnostic messages — across runs, across platforms, and across conforming implementations.

This is a normative requirement. There is no implementation-defined ordering, no thread-dependent scheduling, and no platform-dependent evaluation. Every aspect of simulation is fully determined by the source text and the seed value:

- Storage randomization is derived solely from the seed via a specified PRNG algorithm.
- Clock scheduling uses event-driven `next_toggle_ps` timestamps. Ideal toggle times are tracked as double-precision floats to accumulate sub-picosecond drift fractions, then rounded to integer picoseconds for scheduling (Sections 2.1, 2.2).
- When multiple clocks toggle at the same time, toggle order is determined by declaration order.
- Combinational settling follows a deterministic iteration order (Section 2.6).
- Waveform output records signal values at every tick using exact bit-vector state with no floating-point arithmetic.

If two conforming simulators produce different waveform files for the same source and seed, at least one has a bug.

2.6 Combinational Settling

Combinational settling in `@simulation` follows the same algorithm and rules as `@testbench` (Testbench Specification Section 2.2). Each settling pass evaluates all `ASYNCHRONOUS` blocks across the elaborated hierarchy — including latch transparent-state updates and asynchronous memory reads — and repeats until quiescence or the **100 delta cycle** limit is reached. If the limit is exceeded, the simulator reports a combinational loop runtime error and aborts.

2.5 X and Z Semantics

Simulation follows the same x and z semantics as the JZ-HDL specification (Sections 1.6 and 2.1) and the Testbench Specification (Section 2.5):

- **x is not a runtime value.** It exists only as a compile-time pattern-matching wildcard in CASE/SELECT. It never appears during simulation. All storage initializes with random 0/1 bits, not x.
- **z is a real electrical state** for tri-state nets. Tri-state resolution follows the resolution table in JZ-HDL specification Section 1.6.3, and multi-driver validity is enforced at compile time per Section 1.6.4. If z reaches a non-tristate expression at runtime (despite compile-time structural checks), the simulator reports a runtime error and aborts the current simulation.
- **Registers and latches cannot store z** (JZ-HDL specification Section 1.6.6). Only nets and ports participating in tri-state logic may carry z.

3. SIMULATION STRUCTURE

3.1 Declaration

Syntax:

```
@simulation <module_name>
    @import "<path>";          // optional
    BUS <name> { ... }        // optional

    CLOCK { ... }
    WIRE { ... }
    TAP { ... }

    @new <instance_name> <module_name> { ... }

    @setup { ... }

    // Sequence of @run, @update, and @trace directives
@endsim
```

- <module_name> must refer to a @module in scope.
- CLOCK, WIRE, and TAP blocks define the testbed and monitoring scope.
- @new instantiates the module (identical to @testbench connection rules).

3.2 CLOCK Block (Auto-toggling)

Unlike testbenches, simulation clocks must have defined periods so they can run automatically.

Syntax:

```
CLOCK {
    <clock_id> = { period=<ns> };
    ...
}
```

- <clock_id>: Initializes to 0 and toggles automatically every `period / 2` nanoseconds during @run directives.

3.3 WIRE Block

Syntax: Same as module and testbench WIRE blocks. - Wires act as procedural drivers for module IN ports and observers for OUT ports. - All declared WIRES are automatically included in the waveform output.

3.4 TAP Block (Internal Monitoring)

Syntax:

```
TAP {
    <instance_name>.<signal_name>;
    <instance_name>.<sub_instance>.<signal_name>;
}
```

```
...  
}
```

- **Purpose:** Defines internal RTL signals (registers, wires, memories) to be exposed in the waveform dump.
- **Rules:** Signals must exist in the elaborated hierarchy.
- All TAP signals, along with all CLOCKS and WIRES, are recorded per tick.

4. STIMULUS AND CONTROL DIRECTIVES

4.1 @setup

Syntax:

```
@setup {  
    <wire_id> <= <literal>;  
    ...  
}
```

- Establishes time-zero values for testbench wires.
- Must appear exactly once, immediately after **@new**.

4.2 @run (Time Advancement)

Syntax:

```
@run(ticks=<integer>)  
@run(ns=<number>)  
@run(ms=<number>)
```

- Advances the global simulation time by the specified amount.
- During this time, all declared **CLOCKS** toggle automatically based on their periods.
- Synchronous and asynchronous RTL logic evaluates identically to physical hardware as the clocks toggle.
- You may use only one time unit parameter per **@run** directive.

4.3 @update

Syntax:

```
@update {  
    <wire_id> <= <literal_or_expression>;  
    ...  
}
```

- Applies new stimulus to testbench wires at the exact simulation time it is called (i.e., immediately after the preceding **@run** finishes).
- All assignments within a single **@update** block use **simultaneous assignment semantics**: all RHS expressions are evaluated using the pre-update wire values before any LHS targets are written. This means **b <= a**; **a <= a + 1**; assigns the old value of **a** to **b**, not the incremented value (see Testbench Specification Section 6.5 for details).
- After all assignments complete, combinational logic settles, and the result is logged to the waveform before the next **@run** advances time.

4.4 @run_until (Conditional Time Advancement)

Syntax:

```
@run_until(<signal> == <value>, timeout=<unit>=<amount>)  
@run_until(<signal> != <value>, timeout=<unit>=<amount>)
```

The `@run_until` directive advances simulation time tick-by-tick until the specified condition becomes true, or until the timeout is reached. Clocks toggle automatically during advancement, exactly as with `@run`.

- `<signal>` must be a testbench WIRE identifier (typically connected to a module output).
- `<value>` must be a sized literal.
- The condition is evaluated after each tick (after combinational settling and synchronous domain firing).
- The `timeout` parameter specifies the maximum simulation time to advance. If the condition is not met within the timeout, the simulator reports a **TIMEOUT** runtime error and aborts the simulation.
- The timeout uses the same time units as `@run`: `ns`, `ms`, or `ticks`.

Example:

```
@run_until(count == 8'h05, timeout=ns=1000)
// Simulation advances until count reaches 5, or 1000ns elapses (error)
```

4.5 @run_while (Conditional Time Advancement)

Syntax:

```
@run_while(<signal> == <value>, timeout=<unit>=<amount>)
@run_while(<signal> != <value>, timeout=<unit>=<amount>)
```

The `@run_while` directive advances simulation time tick-by-tick while the specified condition remains true, or until the timeout is reached. It is the logical complement of `@run_until`.

- Same rules as `@run_until` for signal references, value types, and timeout behavior.
- The condition is evaluated after each tick. Simulation stops when the condition becomes false.
- If the timeout is reached while the condition is still true, the simulator reports a **TIMEOUT** runtime error.

Example:

```
@run_while(busy == 1'b1, timeout=ns=5000)
// Simulation advances while busy is high, or 5000ns elapses (error)
```

Rules:

Rule	Description
SIM-030	<code>@run_until</code> / <code>@run_while</code> condition not met within timeout (TIMEOUT runtime error)

4.6 @repeat

Syntax:

```
@repeat <count>
<body>
@end
```

The `@repeat` directive is a **pre-parser text expansion** shared with `@testbench`. Before lexing or parsing, the compiler scans the source text for `@repeat N ... @end` blocks, duplicates the body `N` times, and replaces each standalone occurrence of the identifier `IDX` with the iteration index (0 through `N-1`).

- `<count>` must be a positive integer literal.
- `<body>` may contain any valid simulation content: `@run`, `@update`, comments, or any other text.
- `IDX` is replaced on word boundaries only.
- Nesting is supported.
- `@end` closes only `@repeat` blocks — it does not conflict with `@endsim`.
- `@repeat` inside comments or string literals is ignored.

Example — Burst stimulus with `IDX`:

```
@repeat 4
@update {
    data_in <= 8'hIDX;
}
@run(ns=10)
@end
// Expands to 4 sequential update/run pairs with data_in = 0, 1, 2, 3
```

Rules:

Rule	Description
RPT-001	<code>@repeat</code> requires a positive integer count
RPT-002	<code>@repeat</code> without matching <code>@end</code>

4.7 @print

Syntax:

```
@print("<format_string>", <arg1>, <arg2>, ...)
```

The `@print` directive outputs a formatted message to the simulator's standard output at the current simulation time.

- `<format_string>` is a string literal containing text and optional format specifiers.
- Arguments following the format string are testbench wire identifiers or hierarchical signal references, matched positionally to format specifiers.
- The message is printed after all combinational logic has settled following the most recent `@run`, `@run_until`, `@run_while`, or `@update` directive.
- `@print` may appear anywhere in the simulation body where `@update` is valid.

Format specifiers:

Specifier	Description
<code>%h</code>	Display the argument value in hexadecimal
<code>%d</code>	Display the argument value in decimal

Specifier	Description
<code>%b</code>	Display the argument value in binary
<code>%tick</code>	Display the current tick count (no argument consumed)
<code>%ms</code>	Display the current simulation time in milliseconds (no argument consumed)

- `%tick` and `%ms` are **autonomous specifiers** — they do not consume an argument from the argument list.
- The number of non-autonomous format specifiers must match the number of arguments. A mismatch is a compile error.

Example:

```
@print("wr_ptr = %h at tick %tick", dut.wr_ptr)
@print("time %ms: data_out = %d", data_out)
@print("tick %tick: reset released")
```

4.8 @print_if

Syntax:

```
@print_if(<condition>, "<format_string>", <arg1>, <arg2>, ...)
```

The `@print_if` directive conditionally outputs a formatted message. The message is printed only if `<condition>` evaluates to a non-zero (truthy) value.

- `<condition>` is a testbench wire identifier or hierarchical signal reference. The condition is truthy if any bit is 1.
- The format string and arguments follow the same rules as `@print`.
- `@print_if` may appear anywhere `@print` is valid.

Example:

```
@print_if(full, "FIFO full at time %ms, wr_ptr=%h", dut.wr_ptr)
@print_if(empty, "FIFO empty at tick %tick")
```

Rules:

Rule	Description
PRT-001	Number of non-autonomous format specifiers must match the number of arguments
PRT-002	<code>@print</code> / <code>@print_if</code> may not appear inside <code>@setup</code> or <code>@update</code> blocks

4.9 @trace (Waveform Recording Control)

Syntax:

```
@trace(state=on)
@trace(state=off)
```

The `@trace` directive controls whether signal value changes are recorded to the output waveform file during subsequent `@run`, `@run_until`, and `@run_while` directives.

- **Default state is on.** If no `@trace` directive appears, all signal changes are recorded (backward compatible).
- When `state=off`, the simulation continues to execute — clocks toggle, logic evaluates, time advances — but no signal values are written to the waveform output. This dramatically reduces output file size and simulation time for uninteresting periods (e.g., waiting for a power-on reset counter).
- When `state=on` after a `state=off` period, the simulator writes a **full state snapshot** at the current simulation time before resuming per-tick recording. This ensures the waveform viewer sees correct signal values at the start of the recorded window, not stale time-0 values.
- `@trace` may be toggled any number of times to capture specific windows of interest.
- `@trace` directives apply to all waveform output formats (VCD, FST, JZW).
- For JZW output, each `@trace` toggle writes an annotation of type `trace` to the annotations table, with `message` set to "on" or "off". Viewers can use these annotations to indicate trace-off periods visually.

Example:

```
@setup {
    por    <= 1'b1;
    rst_n  <= 1'b1;
}

// Skip recording during 28ms POR wait
@trace(state=off)
@run(ns=28200000)
@trace(state=on)

// Now record the interesting video output
@run(ns=200000)
```

Rules:

Rule	Description
TRC-001	<code>@trace</code> state must be on or off
TRC-002	<code>@trace</code> may not appear inside <code>@setup</code> or <code>@update</code> blocks

4.10 @mark (Waveform Marker)

Syntax:

```
@mark(<color>)
@mark(<color>, "<message>")
```

The `@mark` directive creates a global marker annotation at the current simulation time. Marks are unconditional — they fire exactly once at the point where they appear in the simulation sequence.

- `<color>` is a color name (one of: RED, ORANGE, YELLOW, GREEN, BLUE, PURPLE, CYAN, WHITE). Case-insensitive in source, stored as uppercase.
- `<message>` is an optional string literal describing the marker.
- Marks are written to the JZW annotations table with `type="mark"`, `signal_id=NULL` (global), and the specified color and message.
- Marks have no effect on VCD or FST output.

Example:

```
@update {
    rst_n <= 1'b1;
}
@mark(GREEN, "Reset released")

@run(ns=100)

@update {
    wr_en <= 1'b1;
}
@mark(BLUE, "Write burst start")

@run(ns=50)
```

Viewer behavior: Marks are displayed as vertical lines at the marked time in the specified color, with an icon at the bottom. Hovering the icon shows the message.

Rules:

Rule	Description
MRK-001	<code>@mark</code> may not appear inside <code>@setup</code> , <code>@update</code> , or <code>@new</code> blocks
MRK-002	Color name must be one of the predefined names

4.11 @alert (Unconditional Alert Annotation)

Syntax:

```
@alert(<color>)
@alert(<color>, "<message>")
```

The `@alert` directive creates a one-shot, unconditional alert annotation at the current simulation time. It is the alert equivalent of `@mark` — it fires exactly once, right where it appears in the simulation sequence.

- `<color>` is a color name (one of: RED, ORANGE, YELLOW, GREEN, BLUE, PURPLE, CYAN, WHITE).
- `<message>` is an optional string literal describing the alert.

- The annotation is written to the JZW annotations table with `type="alert"`, `signal_id=NULL`, and the specified color and message.
- For conditional per-tick alert evaluation, use `@alert_if` inside a `MONITOR` block (§4.14).

Example:

```
@run(ns=1000)
@alert(RED, "Checkpoint reached at 1µs")
@run(ns=1000)
```

Viewer behavior: Alert annotations are displayed as small chevron markers (^) at the bottom of the waveform area at the alert time. Hovering shows the message.

Rules:

Rule	Description
ALT-001	<code>@alert</code> may not appear inside <code>@setup</code> , <code>@update</code> , or <code>@new</code> blocks
ALT-002	Color name must be one of the predefined names

4.12 @mark_if (Conditional One-Shot Marker)

Syntax:

```
@mark_if(<condition>, <color>)
@mark_if(<condition>, <color>, "<message>")
```

The `@mark_if` directive evaluates a condition once at the current simulation time. If the condition is true (non-zero), a mark annotation is created — identical to what `@mark` would produce. If the condition is false, nothing happens.

- `<condition>` is a testbench WIRE identifier or hierarchical signal reference (TAP). If the signal is multi-bit, it is truthy if any bit is 1.
- `<color>` is a color name (one of: RED, ORANGE, YELLOW, GREEN, BLUE, PURPLE, CYAN, WHITE).
- `<message>` is an optional string literal describing the marker.
- Unlike `@alert`, this is **not** registered for per-tick evaluation. It fires at most once, at the point it appears in the simulation sequence.

Example:

```
@run(ns=1000)
@mark_if(dut.error_flag, RED, "Error detected at checkpoint")
@run(ns=1000)
```

Rules:

Rule	Description
MKI-001	<code>@mark_if</code> may not appear inside <code>@setup</code> , <code>@update</code> , or <code>@new</code> blocks

Rule	Description
MKI-002	<code>@mark_if</code> condition must reference a signal in scope (declared in <code>WIRE</code> , <code>CLOCK</code> , or <code>TAP</code>)
MKI-003	Color name must be one of the predefined names

4.13 @alert_if (Conditional One-Shot Alert)

Syntax:

```
@alert_if(<condition>, <color>)
@alert_if(<condition>, <color>, "<message>")
```

The `@alert_if` directive evaluates a condition once at the current simulation time. If the condition is true (non-zero), an alert annotation is created — identical to what a per-tick `@alert` would produce for a single tick. If the condition is false, nothing happens.

- `<condition>` is a testbench `WIRE` identifier or hierarchical signal reference (`TAP`). If the signal is multi-bit, it is truthy if any bit is 1.
- `<color>` is a color name (one of: `RED`, `ORANGE`, `YELLOW`, `GREEN`, `BLUE`, `PURPLE`, `CYAN`, `WHITE`).
- `<message>` is an optional string literal describing the alert.
- Unlike `@alert`, this is **not** registered for per-tick evaluation. It fires at most once, at the point it appears in the simulation sequence.

Example:

```
@run(ns=5000)
@alert_if(dut.fifo_full, RED, "FIFO full at 5ps checkpoint")
@run(ns=5000)
@alert_if(dut.fifo_full, RED, "FIFO full at 10ps checkpoint")
```

Rules:

Rule	Description
ALI-001	<code>@alert_if</code> may not appear inside <code>@setup</code> , <code>@update</code> , or <code>@new</code> blocks
ALI-002	<code>@alert_if</code> condition must reference a signal in scope (declared in <code>WIRE</code> , <code>CLOCK</code> , or <code>TAP</code>)
ALI-003	Color name must be one of the predefined names

4.14 MONITOR Block (Per-Tick Conditional Evaluation)

Syntax:

```
MONITOR {
    @print_if(<condition>, "<format>", ...)
    @mark_if(<condition>, <color>)
    @mark_if(<condition>, <color>, "<message>")
}
```

```

    @alert_if(<condition>, <color>)
    @alert_if(<condition>, <color>, "<message>")
}

```

The MONITOR block defines a set of conditional directives that are evaluated **every tick** during all @run, @run_until, and @run_while directives. Unlike the one-shot forms of @print_if, @mark_if, and @alert_if that appear in the simulation body, directives inside a MONITOR block fire repeatedly whenever their conditions are met.

- At most **one** MONITOR block is permitted per @simulation.
- Only @print_if, @mark_if, and @alert_if are permitted inside a MONITOR block. Other directives (e.g., @run, @update, @print, @mark, @alert) are not allowed.
- The MONITOR block may appear anywhere in the simulation body (before or after @setup, @run, etc.). Its position does not affect when evaluation begins — monitor directives are always evaluated on every tick from the first @run onward.
- Conditions reference WIRE, CLOCK, or TAP signals, following the same resolution rules as one-shot directives.

Example:

```

@simulation fifo_test
    @import "fifo.jz";

    CLOCK { clk = { period=10.0 }; }
    WIRE { rst_n [1]; full [1]; empty [1]; }
    TAP { dut.overflow; }

    @new dut fifo { ... }

    MONITOR {
        @alert_if(dut.overflow, RED, "FIFO overflow detected!")
        @mark_if(full, ORANGE, "FIFO full")
        @print_if(empty, "FIFO empty at %t", $time)
    }

    @setup { rst_n <= 1'b0; }
    @run(ns=50)
    @update { rst_n <= 1'b1; }
    @run(ns=1000)
@endsim

```

Rules:

Rule	Description
MON-001	Only one MONITOR block is permitted per @simulation
MON-002	Only @print_if, @mark_if, and @alert_if are permitted inside MONITOR

Rule	Description
MON-003	MONITOR conditions must reference signals in scope (WIRE, CLOCK, or TAP)

5. CLI USAGE

5.1 --simulate

Files containing @simulation blocks must be run with the --simulate flag. Using --lint or --test on a file that contains @simulation will produce a SIM_WRONG_TOOL error.

```
jz-hdl --simulate sim_file.jz                # produces sim_file.vcd
jz-hdl --simulate sim_file.jz -o output.vcd  # explicit output path
jz-hdl --simulate sim_file.jz --seed=0xCAFE  # reproducible register init
jz-hdl --simulate sim_file.jz --verbose      # print tick resolution, events
jz-hdl --simulate sim_file.jz --jitter=clk:200 # 200ps p-p jitter on clk
jz-hdl --simulate sim_file.jz --jitter=clk_wr:200 --jitter=clk_rd:500
jz-hdl --simulate sim_file.jz --drift=clk:50 # ±50 ppm crystal tolerance
jz-hdl --simulate sim_file.jz --jitter=clk:200 --drift=clk:50 # jitter + drift
jz-hdl --simulate sim_file.jz --tristate-default=GND
```

5.2 Flags

Flag	Description
--simulate	Run all @simulation blocks in the file.
-o <path>	Output waveform file path. Default: <input_basename>.vcd, or the selected format's extension when --fst or --jzw is specified.
--seed=0x<hex>	32-bit seed for register randomization. Default: 0xDEADBEEF.
--vcd	Force VCD output format (default).
--fst	Force FST output format.
--jzw	Force JZW output format (SQLite-based).
--jitter=<clock>:<ps>	Add Gaussian period jitter to a clock. <clock> is the clock name declared in the simulation's CLOCK block. <ps> is the peak-to-peak jitter in picoseconds (= ps/6, clamped at ±ps/2). May be specified multiple times for different clocks. See Section 2.2.1.
--drift=<clock>:<ppm>	Add frequency drift to a clock. <clock> is the clock name declared in the simulation's CLOCK block. <ppm> is the maximum drift in parts per million. The actual drift is selected from a Gaussian distribution (= ppm/3, clamped at ±ppm) at simulation start. May be specified multiple times for different clocks. See Section 2.2.2.
--tristate-default=GND VCC	Apply internal tri-state lowering before simulation, using the selected default when all drivers release.

Flag	Description
--verbose	Print tick resolution, clock periods, and @run/@update events.

6. WAVEFORM OUTPUT

6.1 VCD Format

The simulator produces IEEE 1364 VCD (Value Change Dump) files. The VCD timescale is fixed at `1ns`, and all timestamps are written in nanosecond units regardless of the internal tick resolution. This ensures waveform viewers display readable, properly spaced timestamps.

6.2 Signal Grouping

Signals are organized into VCD scopes:

Scope	Contents
<code>clocks</code>	All <code>CLOCK</code> -declared signals.
<code>wires</code>	All <code>WIRE</code> -declared signals (inputs and observed outputs).
<code><instance></code>	TAP signals, scoped by the hierarchy prefix (e.g., TAP { <code>dut.wr_ptr</code> ; } appears under scope <code>dut</code>).

6.3 What Gets Recorded

Every signal in `CLOCK`, `WIRE`, and `TAP` blocks is sampled and written to the waveform file at every tick during `@run`, and at each `@setup`/`@update` event. The Time 0 dump captures the full initial state before any clock edge.

6.4 JZW Metadata

The JZW format stores simulation metadata in its `meta` table. In addition to the standard keys (`date`, `source_file`, `compiler_version`, `seed`, `tick_ps`, `module_name`), the following keys are written when clock jitter or drift is enabled:

Key	Value	Example
<code>jitter_<clock_name></code>	Peak-to-peak jitter in picoseconds	<code>jitter_clk = "200"</code>
<code>drift_<clock_name></code>	Maximum drift in ppm (configured value)	<code>drift_clk = "50.0"</code>
<code>drift_actual_<clock_name></code>	Actual selected drift in ppm	<code>drift_actual_clk = "8.027143"</code>

One `jitter_` key is written per jittered clock. Two `drift_` keys are written per drifted clock: one for the configured maximum and one for the actual selected value. Clocks without jitter or drift have no corresponding keys. This allows waveform viewers and analysis tools to determine whether jitter/drift was active and with what parameters, and to reconstruct the exact timing behavior of each clock.

7. EXAMPLE

```
@simulation fifo_ctrl
    @import "fifo.jz";

    // Define clocks with different frequencies
    CLOCK {
        clk_wr = { period=10.0 }; // 100 MHz
        clk_rd = { period=25.0 }; // 40 MHz
    }

    WIRE {
        rst_n [1];
        wr_en [1];
        rd_en [1];
        data_in [8];
        data_out [8];
        full [1];
        empty [1];
    }

    // Monitor internal FIFO pointers
    TAP {
        dut.wr_ptr;
        dut.rd_ptr;
    }

    @new dut async_fifo {
        clk_wr [1] = clk_wr;
        clk_rd [1] = clk_rd;
        rst_n [1] = rst_n;
        wr_en [1] = wr_en;
        rd_en [1] = rd_en;
        din [8] = data_in;
        dout [8] = data_out;
        full [1] = full;
        empty [1] = empty;
    }

    MONITOR {
        @alert_if(full, RED, "FIFO full")
        @mark_if(empty, CYAN, "FIFO empty")
    }

    @setup {
        rst_n <= 1'b0;
        wr_en <= 1'b0;
```



```

        rd_en    <= 1'b0;
        data_in  <= 8'h00;
    }

    // Hold reset for 50ns while clocks run
    @run(ns=50)

    // Release reset
    @update {
        rst_n <= 1'b1;
    }

    // Wait for internal synchronization
    @run(ns=50)

    // Burst write data
    @update {
        wr_en    <= 1'b1;
        data_in  <= 8'hAA;
    }
    @run(ns=10)

    @update {
        data_in <= 8'hBB;
    }
    @run(ns=10)

    @update {
        wr_en <= 1'b0;
        rd_en <= 1'b1;
    }

    // Run long enough to see read clocks process the data
    @run(ns=100)
@endsim

```